Game Server & Client Programmer Portpolio

Autor : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738

PIP Project

프로젝트 개요

프로젝트 이름	PIP Project
장르	Action RPG (DirectX 12 + Jolt Physics)
설명	프레임 단위 정밀 타격을 위한 액션 RPG
담당 역할	Server Physics & AI Sync
기술 스택	C++23 · Jolt Physics · Behavior Tree · IOCP

Behavior Tree(BT)를 활용한 보스 AI

Problem: 보스의 복잡한 페이즈(Phase) 변화, 스킬, 타겟팅 로직이 얽혀 스파게티 코드 가능성

Solution: 행동 트리(BT)의 Action 노드를 상속 구조로 모듈화하고, 타겟(유저) 상실 시 즉시 대응하는 예외처리(안전장치) 노드를 Decorator로 배치

Performance Gain: 보스 패턴 로직의 재사용성 확보

```
BTBuilder builder;
auto root = builder
    .sequence()
    .leaf<Condition_IsAlive>()
    .leaf<Condition_IsHitted>(DecoratorType::Inverter)
    .selector() // 전체 로직 우선순위 결정
        // --- [우선순위 1] 전투 로직 ---
        .sequence()
            .leaf<Condition_HasEnemy>() // 타겟(적군)이 있는가?
            .leaf<Condition_IsTargetInLeashRange>(25.0f) // 25m 내에 있으면 추격 유지
            .selector()
            // 1-1. 강력한 공격 시도 (사거리 2.5m)
                .sequence()
                    .leaf<Condition_IsEnemyInRange>(1.5f)
                    .leaf<Action_AttackEnemy>(heavyAtk)
                .end()
            // 1-2. 일반 공격 시도 (사거리 1.5m)
            .sequence()
                .leaf<Condition_IsEnemyInRange>(1.5f)
                .leaf<Action_AttackEnemy>(normalAtk)
            .end()
            // 1-3. 타겟 사거리 밖이면 네비메쉬 추격
            .sequence()
                .leaf<Action_UpdateEnemyPosToTarget>()
                .leaf<Action_FindPath>("MainStage_NavMesh")
                .leaf<Action_FollowPath>(6.0f) // 추격 속도 6.0
```

서버 물리 엔진(Jolt) 부하 억제 설계

Problem: 다수의 플레이어와 몬스터 객체 물리 연산을 서버가 모두 담당할 경우 심각한 물리 스레드 병목 발생

Solution: 대기 중인 NPC의 Motion Type을 Kinematic/Static(Sleep 상태)으로 설정하여 연산에서 제외하고, 교전 시 Dynamic으로 자동 승격(Dynamic Promotion)

Performance Gain: 충돌 연산 부하 최소화를 통한 안정적 서버 물리 프레임 타임 확보

정밀 피격 동기화를 위한 지연 보상 (Rewind)

Problem: 네트워크 지연(Ping)으로 인해 클라이언트 화면의 몬스터 위치와 서버 위치가 달라 타격감 훼손

Solution: 모든 물리 객체의 최근 1초간 위치/히트박스 스냅샷을 원형 큐에 저장. 공격 패킷 타임스탬프 기반 과거 시간으로 Rewind 후 판정 처리

Performance Gain: Ping 100ms 환경에서도 불일치 없는 서버 피격 판정 달성

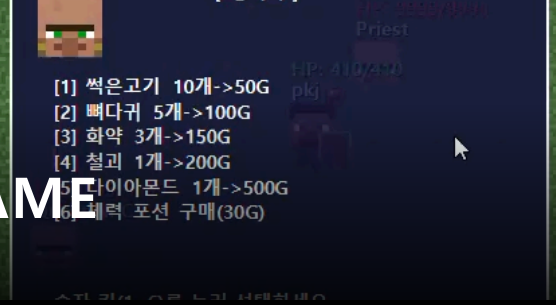
Data Race가 없는 안전한 Room(인스턴스 던전) 아키텍처

Problem: 한 방(Room)에 속한 객체 상태를 여러 워커 스레드가 동시 다발적으로 접근하여 수정하면 데이터 오염 및 데드락 위험

Solution: 방(Room)마다 고유한 Job Queue(명령 대기열)를 부여하여, 상태를 변경하는 모든 로직(이벤트)을 큐에 밀어넣고 단일 스레드 흐름으로 순차 처리(직렬화)

Performance Gain: 객체 단위의 동기화 락(Lock) 비용 완전 제거로 인해 서버 반응 속도 향상 및 데드락 100% 방지, MO RPG 특성을 이용

SImplest Minecraft MMORPG GAME



프로젝트 개요

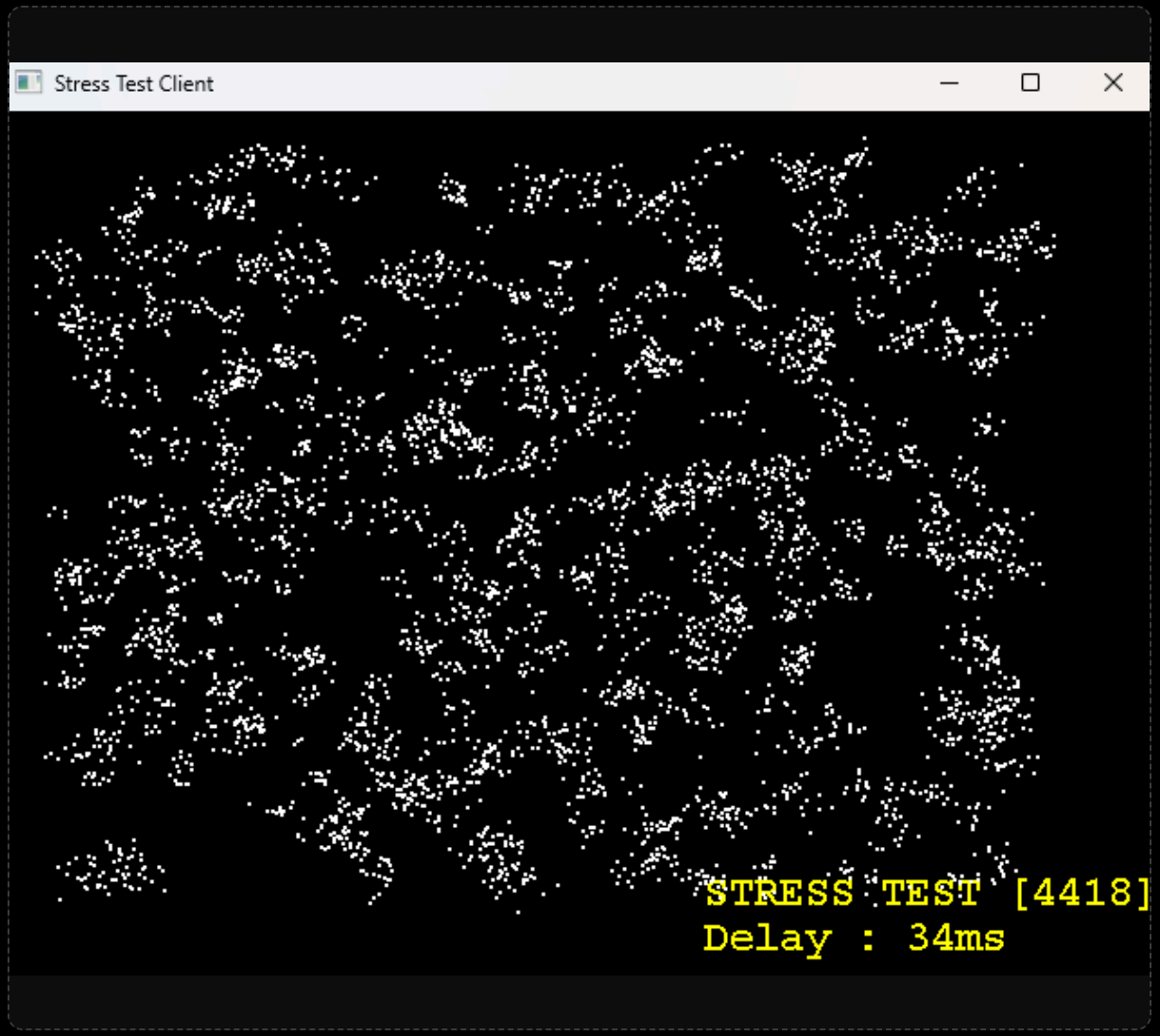
프로젝트 이름	ServerGP (겜서버 텀프로젝트)
장르	MMORPG server, client
설명	IOCP 기반 C++ 멀티스레드 서버. 동접 4400명 및 단일 맵 20만 객체 동시 구동
담당 역할	Server Architecture & Optimization, client dev
기술 스택	C++ IOCP ODBC Lock-Free Queue Lua

시야 처리 병목 최적화 (Sector Grid)

Problem: 20만 마리의 NPC와 유저가 서로의 거리를 O(N^2)로 계산하여 서버 틱 심각한 저하 발생

Solution: 공간을 고정 크기의 Grid(Sector)로 분할하고 std::shared_mutex(Read-Write Lock)를 적용하여 인접 섹터의 객체만 탐색하도록 변경

Performance Gain: 탐색 복잡도를 O(k) 수준으로 낮추어 동시 접속자 수 4400명 안정적 유지



네트워크 I/O 병목 및 데이터 프래그멘테이션 최적화

Problem: 매 틱마다 쏟아지는 수많은 객체의 이동/투사체 패킷 전송(Send) 시스템 콜 비용 과다

Solution: 전송 큐에 패킷을 버퍼링 후 WSABUF 배열을 이용한 Scatter-Gather I/O(Batching)로 뭉쳐서 보내고, 수신단에서 링 버퍼 기반 재조립(Reassembly) 파서 구현

Performance Gain: WSASend 시스템 콜 횟수 대폭 감소 및 패킷 유실률 제로 달성

```cpp
// 1. 송신 버퍼링 및 Scatter-Gather용 WSABUF 배열 세팅
EXP_OVER(std::vector<std::vector<char>>&& data)
    : type(io_type::send), pending_data(std::move(data))
{
    ZeroMemory(&over, sizeof(over));
    wsa_bufs.reserve(pending_data.size());

    // 버퍼 복사 없이 포인터와 크기만 배열로 엮기 (Scatter-Gather)
    for (auto& d : pending_data) {
        wsa_bufs.push_back({ static_cast<ULONG>(d.size()), d.data() });
    }
}

// 2. 비동기 락프리 기반 전송 (Flush)
void SESSION::flush_send() {
    std::vector<std::vector<char>> sending_data;
    {
        std::lock_guard<std::mutex> lock(send_mtx);
        sending_data.swap(send_queue); // 큐에 쌓인 패킷들 한 번에 들고오기
    }

    EXP_OVER* send_over = new EXP_OVER(std::move(sending_data));
    DWORD send_bytes;

    // 모아둔 WSABUF 배열을 한 번의 시스템 콜로 일괄 전송
    WSASend(client_, send_over->wsa_bufs.data(),
            static_cast<DWORD>(send_over->wsa_bufs.size()),
            &send_bytes, 0, &send_over->over, NULL);
}
```

DB 스레드 분리 및 Stored Procedure 설계

Problem: ODBC DB 접근(동기식) 시 IOCP 워커 스레드가 블로킹되어 서버 응답 불능(Starvation) 발생 위험

Solution: DB 전담 스레드 풀을 분리하고 Lock-Free Queue로 작업을 위임. 복잡한 쿼리는 DB의 Stored Procedure(저장 프로시저)로 호출해 통신 및 쿼리 비용 최소화

Performance Gain: 워커 스레드의 I/O 블로킹 완전 제거로 동접 5,000명 환경에서도 지연 없는 즉각적 응답 보장

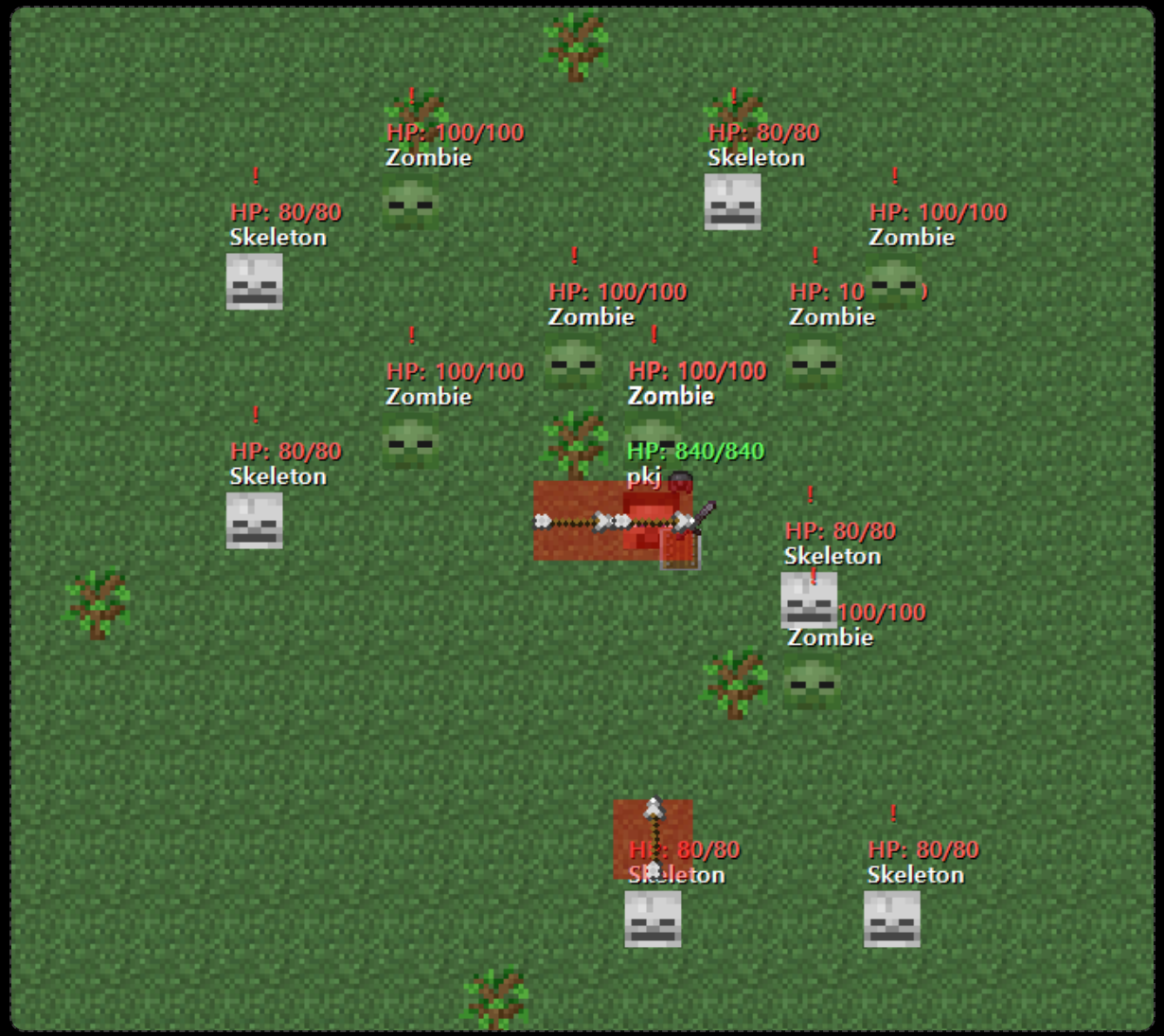
```
// ① IOCP 워커: DB 접근 없이 큐에 위임
DBManager::Instance().PushTask(task);   // Lock-Free Queue, 즉시 리턴

// ② DB 전담 스레드: 큐 소비 + Stored Procedure 실행
void DBManager::DBWorkerThread() {
    while (m_db_thread_running) {
        DBTask task;
        if (m_task_queue.try_pop(task)) {
            std::wstring query = L"EXEC check_auth '" + w_uid + L"'";
            SQLExecDirect(hstmt, (SQLWCHAR*)query.c_str(), SQL_NTS);
            m_notifyCb(task.session_id, task.ioType, task);  // 결과를 IOCP로 반환
        }
    }
}

// ③ 결과를 PQCS로 완료 포트에 게시 → 워커 스레드가 비동기 수신
PostQueuedCompletionStatus(h_iocp, 1, session_id, &over->over);
```

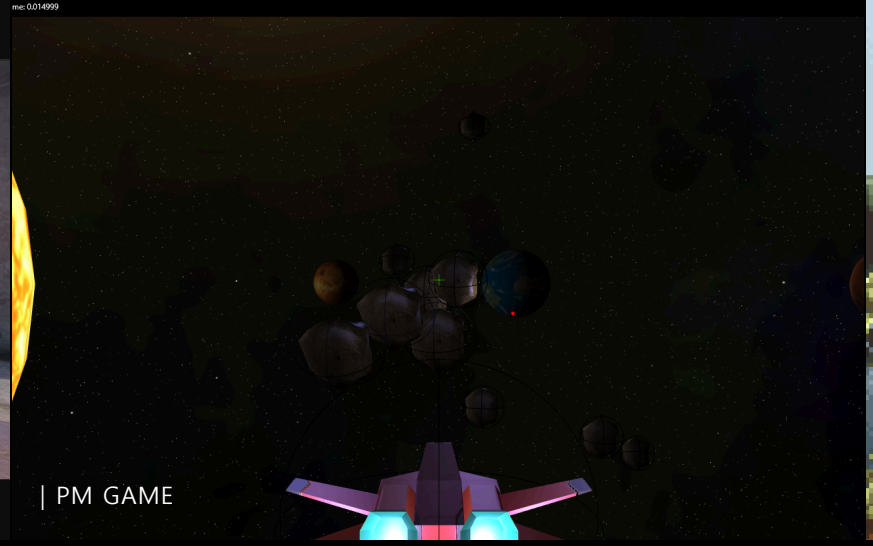
Lua Script를 활용한 확장성 있는 NPC AI

Problem: 20만 객체의 모든 AI 패턴을 하드코딩하기 어렵고, 공유된 Lua State에 여러 스레드가 접근하면 심각한 락(Lock) 병목 발생

Solution: C++ 서버에서 `thread_local` 기반으로 워커 스레드마다 독립적인 `lua_State` 풀을 유지하여, 상호 배제(Lock) 비용 없이 AI 스크립트 고속 실행

Performance Gain: 하드코딩 탈피 및 동기화 오버헤드 0%를 달성하여 방대한 AI 처리량 확보





PROJECTS

게임 엔진 프로젝트 unity 개발 프로젝트

- unity 기능을 최대한 활용한 게임 개발
- 스플라인, 프리팹, 콜리전, 콜라이더, Scene Hirachy

PM GAME 컴퓨터 그래픽스

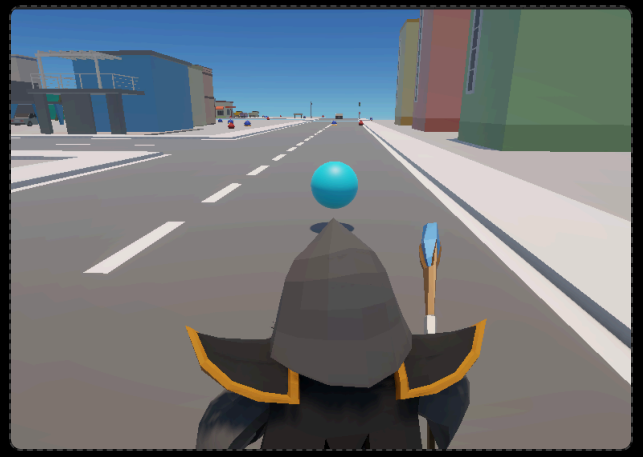
- 컴퓨터 그래픽스 및 OpenGL 사용한 게임개발
- 고급 텍스쳐링 기법 사용, 직접광, 스포트 라이트, 점 조명 구현
- 씬, 메인루프, 입력, 오브젝트 생성 삭제, 그래픽스 엔진, glsl, 기본적인 3D 자체 게임 엔진

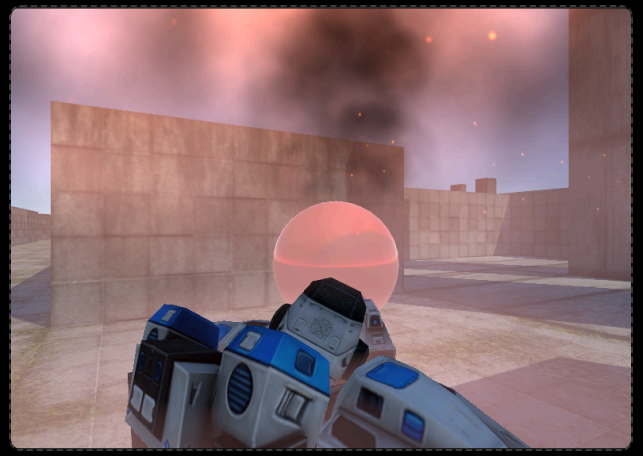
메탈 슬러그 모작 windows programming

- win32사용한 2D 게임 개발
- 충돌체와 이동 속도 계산을 위한 정밀한 물리 도입
- 2D 애니메이션을 위한 세밀한 상태머신 개발

Unity Project

프로젝트 개요



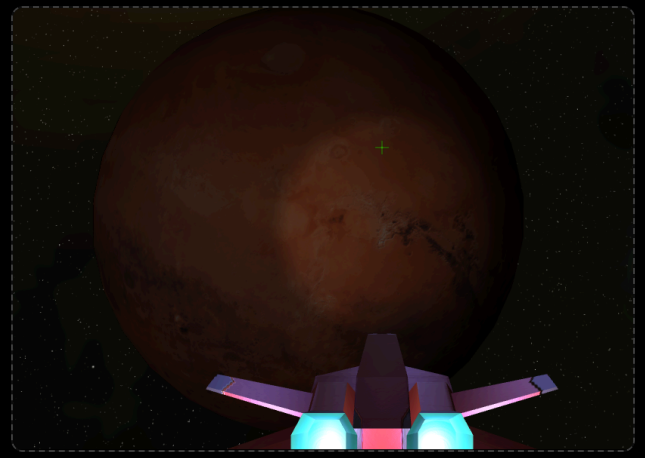


프로젝트 이름	Unity Project
장르	TPS,레이싱 등
설명	유니티 6.0을 사용한 여러 프로젝트 (5개)입니다
개발 인원	1명
담당 역할	클라이언트 개발
개발 언어	untiy6, c#
사용 툴 & 라이브러리	Visual Studio, Git, unity
제작 기간	2025 / 05 ~ 2025 / 06 (한달)

유튜브 링크

PM GAME | 3D Space Shooting Game

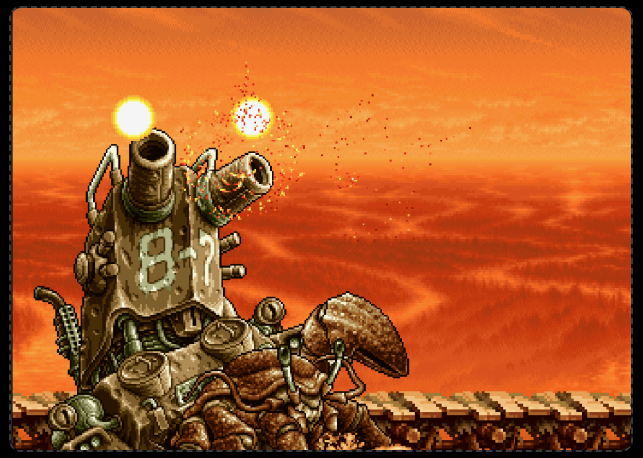
프로젝트 개요



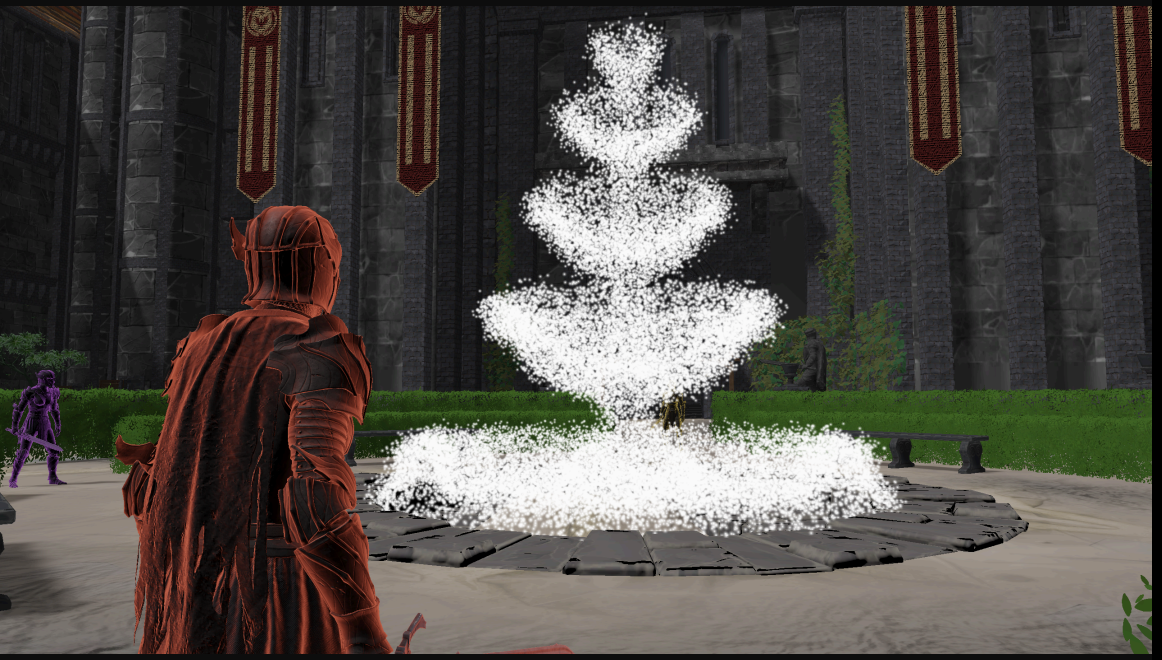
프로젝트 이름	PM GAME
장르	3D 1/3인칭 슈팅게임
설명	3D 1/3인칭 우주선 슈팅 게임, 날아오는 운석을 파괴하라
개발 인원	2명
담당 역할	그래픽스, 오브젝트 관리, 게임 메인 루프, 입력, 사운드, 조명, 적 로직, 플레이어 로직 등 거의 모든 클라이언트 개발 및 자체 엔진 개발 제작
개발 언어	C++20
사용 툴 & 라이브러리	Visual Studio, Git, OpenGL, GLM, fmod
제작 기간	2024 / 10 ~ 2024 / 11

Metal Slug 모작

프로젝트 개요



프로젝트 이름	메탈슬러그 모작
장르	2D 플랫포머 슈팅
설명	메탈슬러그 게임을 win32라이브러리를 사용해서 모작을 만들었습니다.
개발 인원	2명
담당 역할	애니메이션 스프라이트 편집, 상태머신 작성, 플레이어 로직, 보스와 적의 공격 로직 모든 리소스 관리,편집
개발 언어	C++20, win32, fmod
사용 툴 & 라이브러리	Visual Studio, Git
제작 기간	2024 / 05 ~ 2024 / 06



Thank You

포트폴리오를 검토해 주셔서 감사합니다.

Author : 박경준

E-Mail : pkj7323@gmail.com

HP : 010-4674-2738